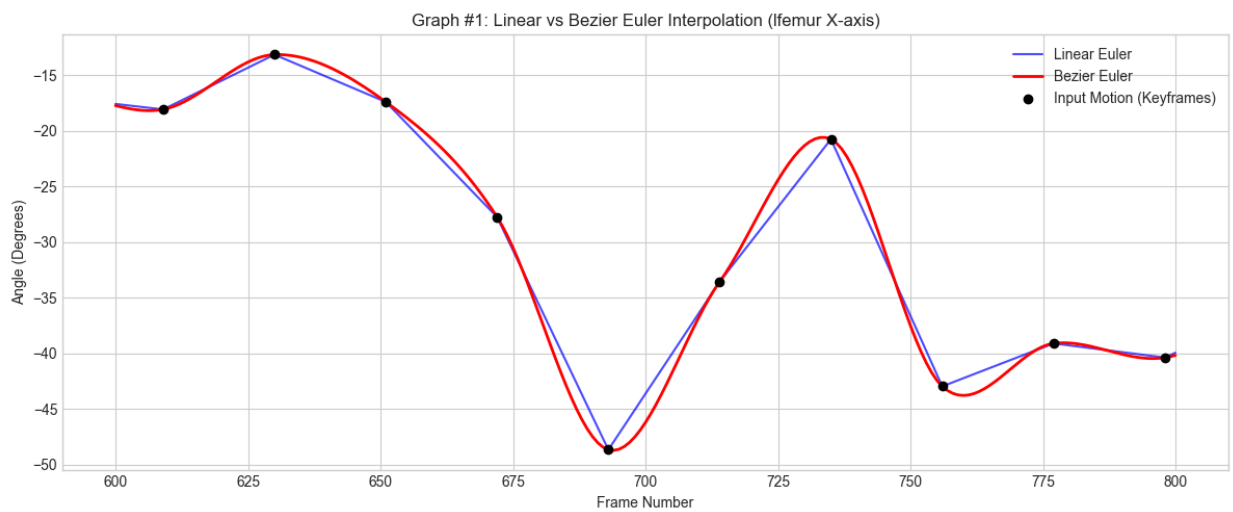
CSCI 520 Assignment 2: Motion Capture Interpolation Report

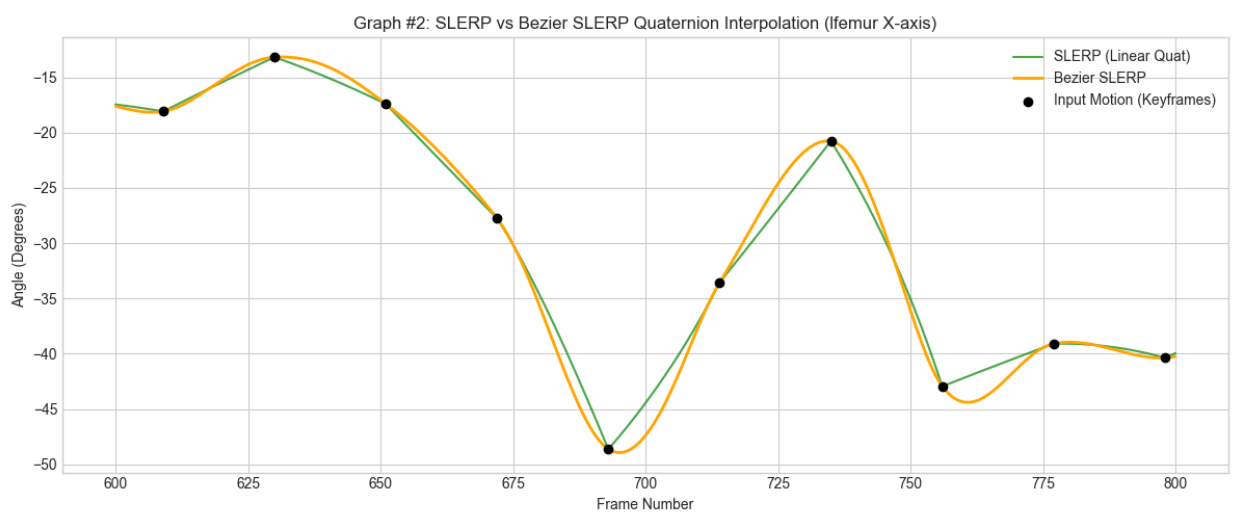
Richard Wang

ywang058@usc.edu

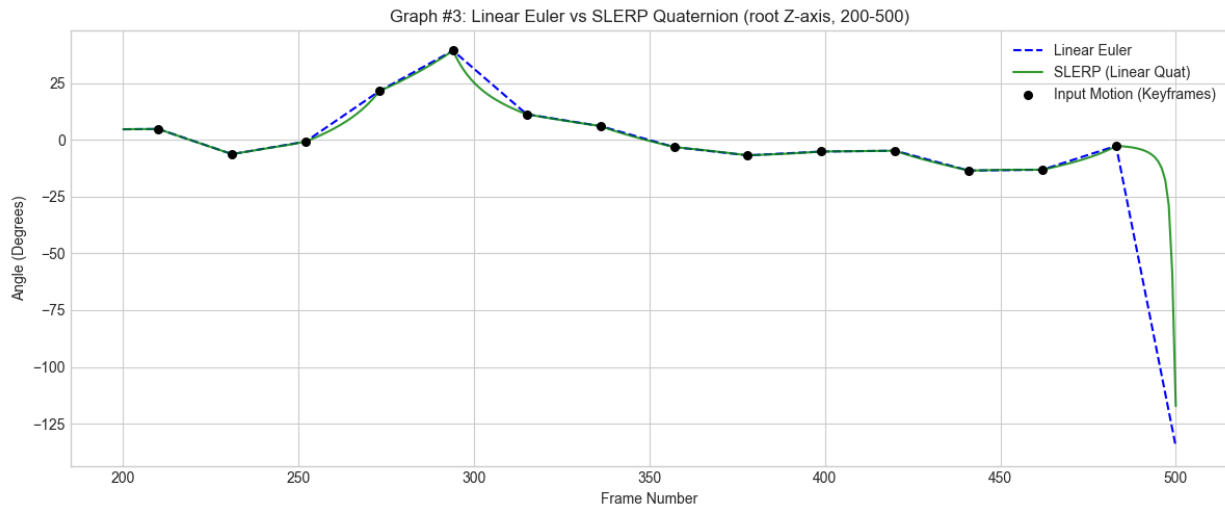
1. Plot four graphs as part of the report (use any program you want, for example, Matlab, GNU Octave, GNU Plot, etc.). For each of these graphs, the X axis should be the frame number, and Y axis the angle in degrees. When using quaternion interpolation, convert the quaternion results back to Euler angles to plot the graphs. **All graphs should show three curves: the two techniques being compared, and the input motion.**

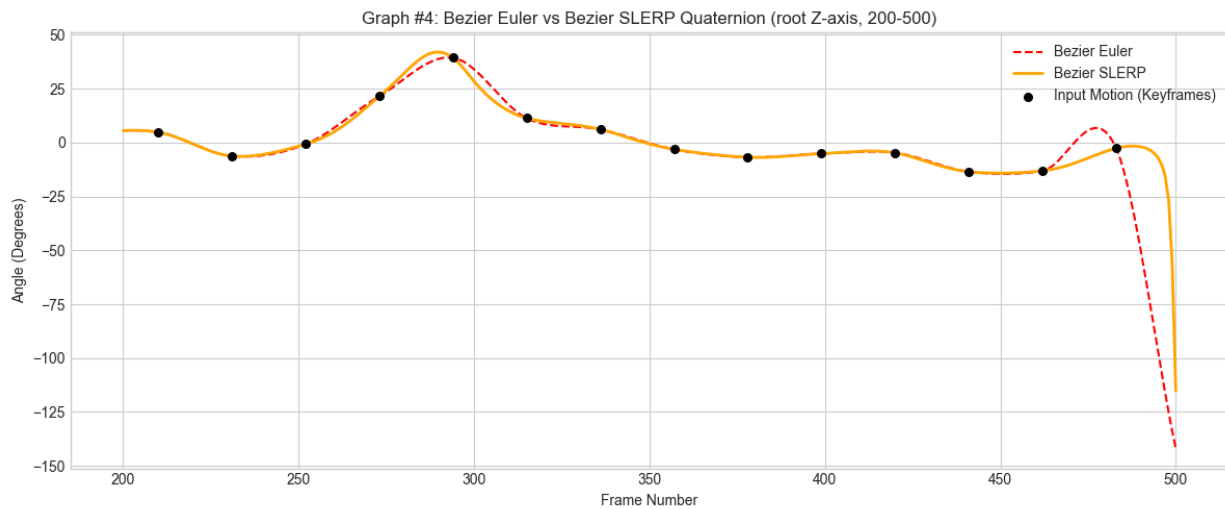
   - Graph #1 compares **linear Euler** to **Bezier Euler** interpolation (and input).
     Graph #2 compares **SLERP quaternion** to **Bezier SLERP quaternion** interpolation (and input).
     Plot graphs #1, #2 for **lfemur** joint, rotation around **X** axis, frames **600-800**, for **N=20**, for **131_04-dance.amc** input file.





- Graph #3 compares **linear Euler** to **SLERP** quaternion (and input).
  Graph #4 compares **Bezier Euler** to **Bezier SLERP quaternion** (and input).
  Plot graphs #3, #4 for **root** joint, rotation around **Z** axis, frames **200-500**, for **N=20**, for **131_04-dance.amc** input file.





2. Three videos demonstrating your results (all three for 135_06-martialArts.amc, N=40):

   - Input motion and Bezier Euler (superimposed on top of each other)
     - Appendix file: **01_135_06-martialArts_Bezier_Euler_40.mp4**
   - Input motion and SLERP quaternion (superimposed on top of each other)
     - Appendix file: **02_135_06-martialArts_Slerp_Quaternion_40.mp4**
   - Input motion and Bezier SLERP quaternion (superimposed on top of each other)
     - Appendix file: **03_135_06-martialArts_Bezier_Quaternion_40.mp4**

3. In addition to the graphs and the videos, your report should contain a narrative giving your findings and observations. Which techniques did you successfully implement? How well does each of the techniques work? What are the strengths and weaknesses of each technique? Include any additional findings, interesting observations or insights gained during this homework. Also, describe any extra credit implemented.

Linear Euler, Bezier Euler, SLERP quaternion, and Bezier SLERP quaternion are successfully implemented in this assignment in the **`Interpolator`** class:

```
// interpolation routines
void LinearInterpolationEuler(Motion * pInputMotion, Motion * pOutputMotion, int N);
void BezierInterpolationEuler(Motion * pInputMotion, Motion * pOutputMotion, int N);
void LinearInterpolationQuaternion(Motion * pInputMotion, Motion * pOutputMotion, int N);
void BezierInterpolationQuaternion(Motion * pInputMotion, Motion * pOutputMotion, int N);
```

The baseline **Linear Euler** performs the worst, which shows some sudden, funny, unnatural rotation jumps as observed in the video (around 0:13s in *01_135_06-martialArts_Bezier_Euler_40.mp4*). These artifacts may be caused by the gimbal lock interpolation singularity issue. Although **Bezier Euler** achieves a smoother interpolation across keyframes, the sudden big rotation artifacts still exist. **SLERP quaternion**, on the other hand, resolves the gimbal lock sudden rotation issue, resulting in a more stable motion path, though the transitions between keyframes remain noticeably mechanical comparing to the Bezier methods. At last, the **Bezier SLERP quaternion** technique provides the most sophisticated results; it not only benefits from the Bezier method's high-order path smoothing but also avoids the gimbal lock issue.

Highlights & Extra Credit Features

1) **Unified Interpolation Architecture (Refactoring):** I refactored the codebase to use a unified ReducedPosture struct instead of separate Euler and Quaternion versions. This optimized memory usage and allowed for a cleaner implementation of the De Casteljau algorithm that works generically across different interpolation modes

```
///@note We don't use the entire Posture struct for interpolation, since bone_translation and bone_length are unused.
///      We create a reduced posture struct that only contains what we need for interpolation.
struct ReducedPosture
{
    vector root_pos;
    vector bone_rotation[MAX_BONES_IN_ASF_FILE];

    ReducedPosture(const Posture& p) {
        root_pos = p.root_pos;
        std::copy(std::begin(p.bone_rotation), std::end(p.bone_rotation), std::begin(bone_rotation));
    }

    ReducedPosture() = default;

    static ReducedPosture InterpolateEuler(double t, const ReducedPosture& start, const ReducedPosture& end) {
        ReducedPosture res;
        res.root_pos = Interpolator::Lerp(t, start.root_pos, end.root_pos);
        for (int i = 0; i < MAX_BONES_IN_ASF_FILE; ++i) {
            res.bone_rotation[i] = Interpolator::Lerp(t, start.bone_rotation[i], end.bone_rotation[i]);
        }
        return res;
    }

    static ReducedPosture InterpolateQuaternion(double t, const ReducedPosture& start, const ReducedPosture& end) {
        ReducedPosture res;
        res.root_pos = Interpolator::Lerp(t, start.root_pos, end.root_pos);
        for (int i = 0; i < MAX_BONES_IN_ASF_FILE; ++i) {
            res.bone_rotation[i] = Interpolator::Slerp(t, start.bone_rotation[i], end.bone_rotation[i]);
        }
        return res;
    }
};
```

2) **Generalized De Casteljau Implementation:** I implemented a version that operates natively in quaternion space using successive SLERP operations. This helper function abstract and unified the usage of interpolations.

```
Input: vector
Process: Lerp
Output: vector
```

```
vector Interpolator::DeCasteljauEuler(double t, vector p0, vector p1, vector p2, vector p3)
{
    auto q0 = Lerp(t, p0, p1);
    auto q1 = Lerp(t, p1, p2);
    auto q2 = Lerp(t, p2, p3);
    auto r0 = Lerp(t, q0, q1);
    auto r1 = Lerp(t, q1, q2);
    auto result = Lerp(t, r0, r1);
    return result;
}
```

```
Input: quaternion
Process: Slerp
Output: quaternion
```

```
Quaternion<double> Interpolator::DeCasteljauQuaternion(double t, const Quaternion<double>& p0, const Quaternion<double>&
p1, const Quaternion<double>& p2, const Quaternion<double>& p3)
{
    auto q0 = Slerp(t, p0, p1);
    auto q1 = Slerp(t, p1, p2);
    auto q2 = Slerp(t, p2, p3);
    auto r0 = Slerp(t, q0, q1);
    auto r1 = Slerp(t, q1, q2);
    auto result = Slerp(t, r0, r1);
    return result;
}
```

```
Input: vector
Process: convert vector to quaternion -> Slerp -> convert quaternion to vector
Output: vector
```

```
vector Interpolator::DeCasteljauQuaternion(double t, const vector& v0, const vector& v1, const vector& v2, const vector&
v3)
{
    Quaternion<double> p0, p1, p2, p3;
    Euler2Quaternion(v0.p, p0);
    Euler2Quaternion(v1.p, p1);
    Euler2Quaternion(v2.p, p2);
    Euler2Quaternion(v3.p, p3);

    auto q0 = Slerp(t, p0, p1);
    auto q1 = Slerp(t, p1, p2);
    auto q2 = Slerp(t, p2, p3);
    auto r0 = Slerp(t, q0, q1);
    auto r1 = Slerp(t, q1, q2);
    auto qResult = Slerp(t, r0, r1);

    vector vResult;
         Quaternion2Euler(qResult, vResult.p);
         return vResult;
}
```